

Требования к диаграммам

Диаграммы, которые должны быть в курсовом:

- диаграмма вариантов использования;
- логическая схема базы данных;
- архитектура приложения / диаграмма компонентов;
- диаграмма классов;
- блок-схема алгоритма ... (название, например, отправки сообщения; проверки страхования автомобиля; оформления заказа и тд);
- диаграмма последовательности ... (нужно уточнять чего, например, добавления товара; оформления заказа и тд).

В файле «Требования_к_курсовому_поООП,_ООТСП,СТПМУ_2022» есть информация как оформлять схемы.

Диаграмма вариантов использования:

- все чертежи должны быть черно-белыми;
- должна присутствовать граница системы (граница системы – большой прямоугольником, внутри которого размещаются варианты использования, с названием системы сверху);
- диаграмму делаем в draw.io / в приложении Microsoft Visio или в другом приложении (никакой автогенерации);
- только прямые линии к вариантам использования, никаких скруглений, углов;
- для текста вариантов использования не «добавить..», а «добавление..» (не глаголы, а существительные);
- обязательно использовать <<Включить>> / <<Расширить>>
- 1 функция – 1 вариант использования (1 овал), не надо 2 функции писать в одном овале;
- все овалы должны быть одинакового размера.

Соединительные линии диаграммы вариантов использования показаны на рисунке 1.

Отношение ассоциации: _____

Отношение обобщения: _____>

Отношение включения: -----<<включить>>----->

Отношение расширения: -----<<расширить>>----->

Рисунок 1 – Соединительные линии

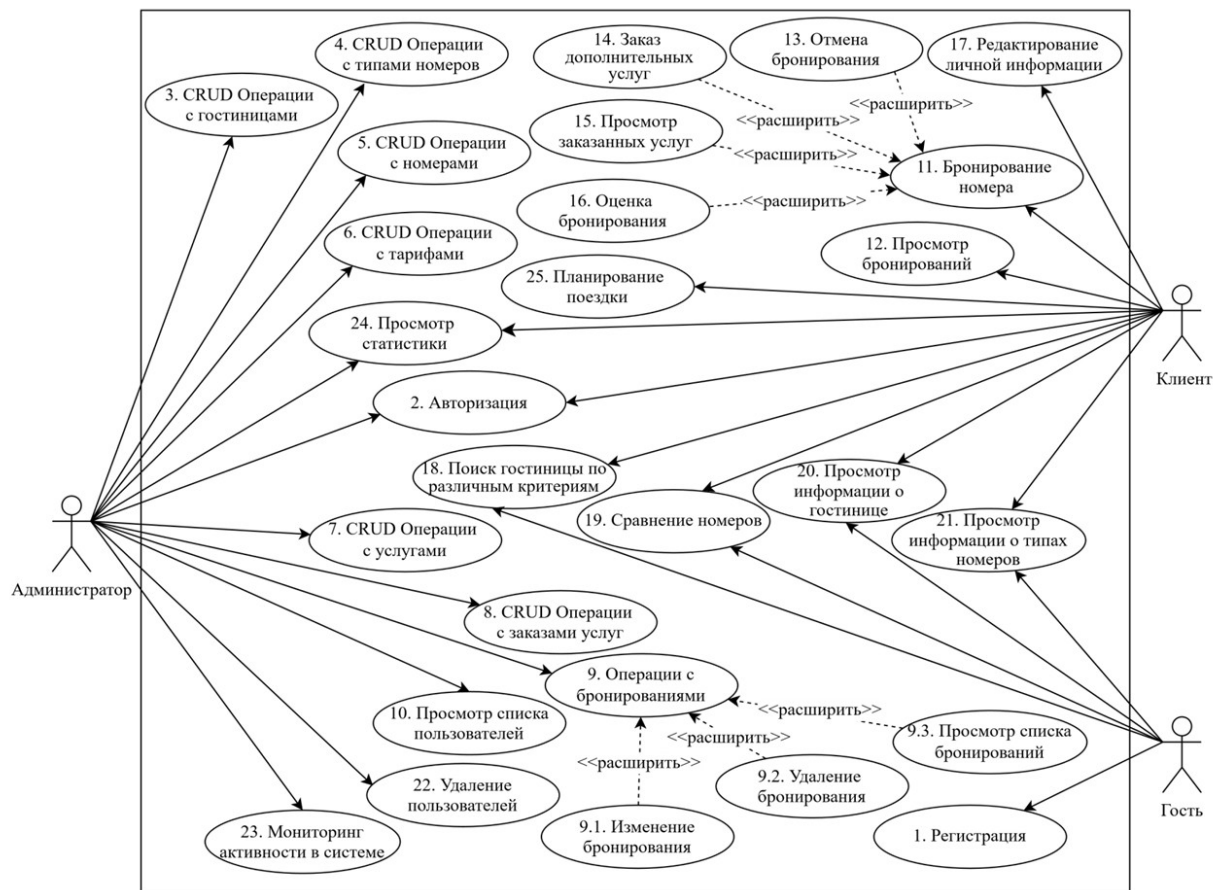


Рисунок 2 – Пример диаграммы вариантов использования

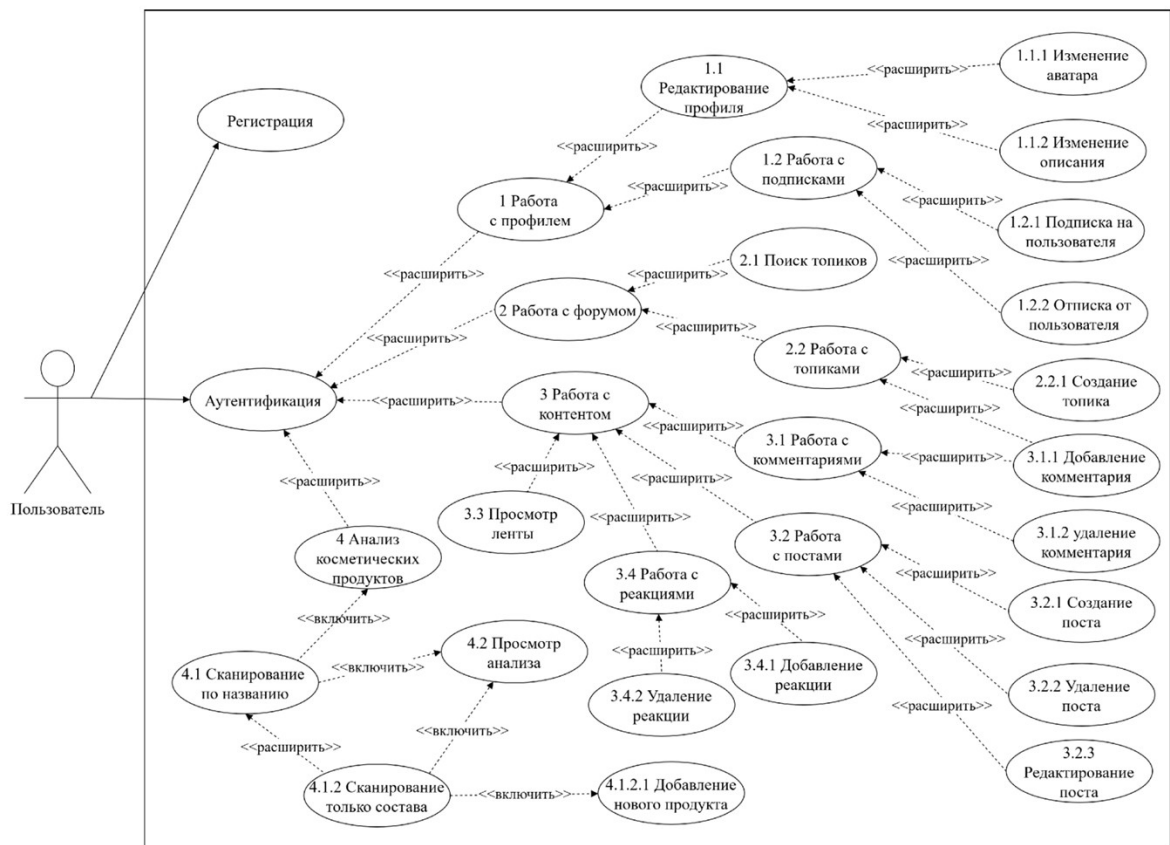


Рисунок 3 – Пример диаграммы вариантов использования

Логическая схема базы данных:

Если таблица имеет большое количество полей, допускается показывать только наиболее существенные.

Статические связи между таблицами показываются сплошными линиями, конечные точки которых обозначают следующая кратность:

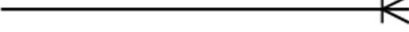
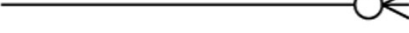
	Один
	Много
	Только один
	Ноль или 1
	Один или много
	Ноль или много

Рисунок 4 – Связи между таблицами

Допускается представлять физическую модель данных. Она содержит все детали, необходимые для создания данных: наименования таблиц и столбцов, типы данных, определения первичных и внешних ключей и т.п. Физическая модель строится на основе логической с учетом ограничений.

Пересечений линий на диаграмме быть не должно. Необходимо разместить таблицы так, чтобы связи между ними не пересекались. Если такой возможности нет, на пересечении необходимо использовать «мост», как показано на рисунке 5. В draw.io называется «Перемычка линий» и находится во вкладке «Стиль» при нажатии на линию.

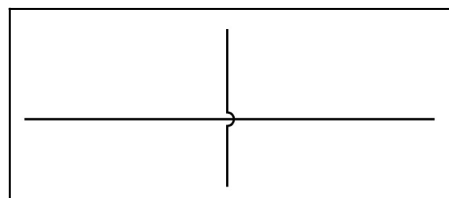


Рисунок 5 – Пример перемычки линий

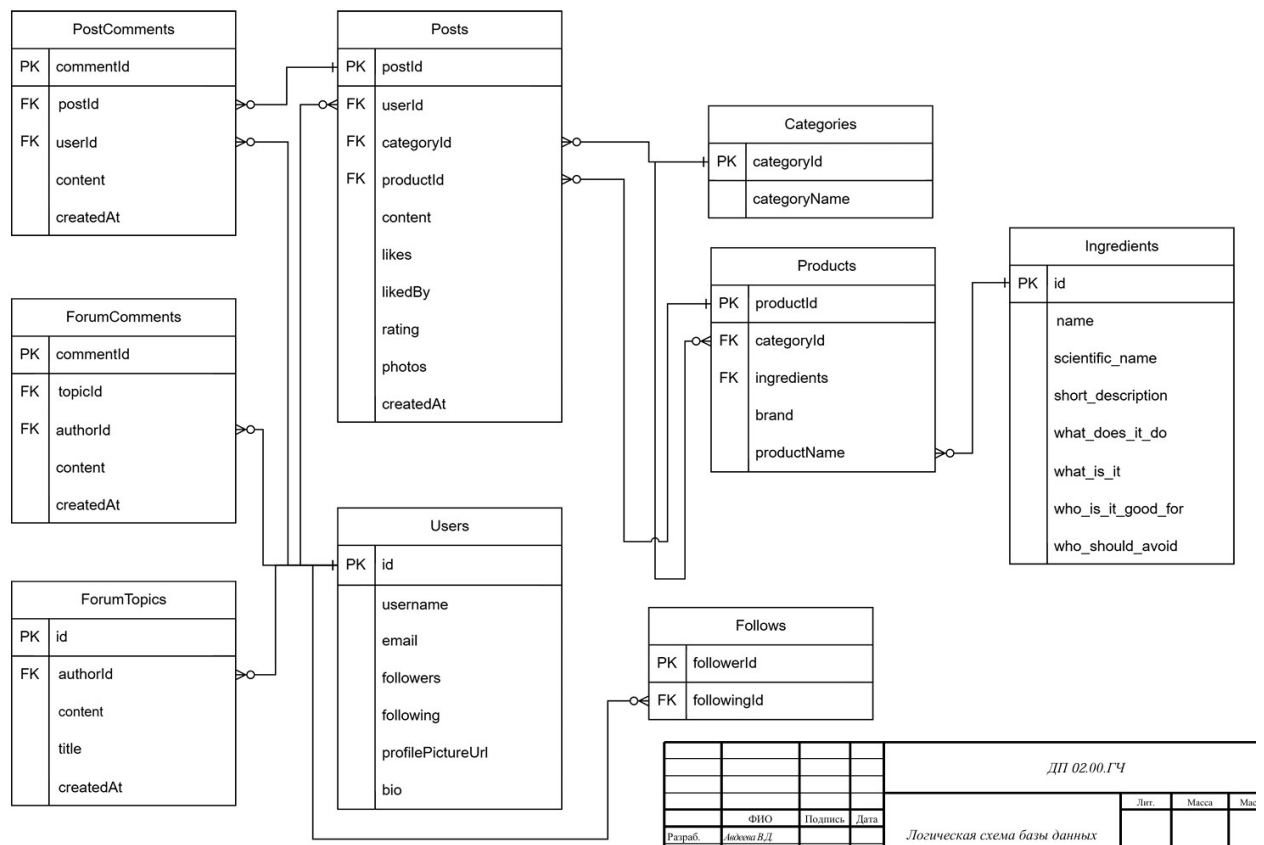


Рисунок 6 – Пример логической схемы базы данных

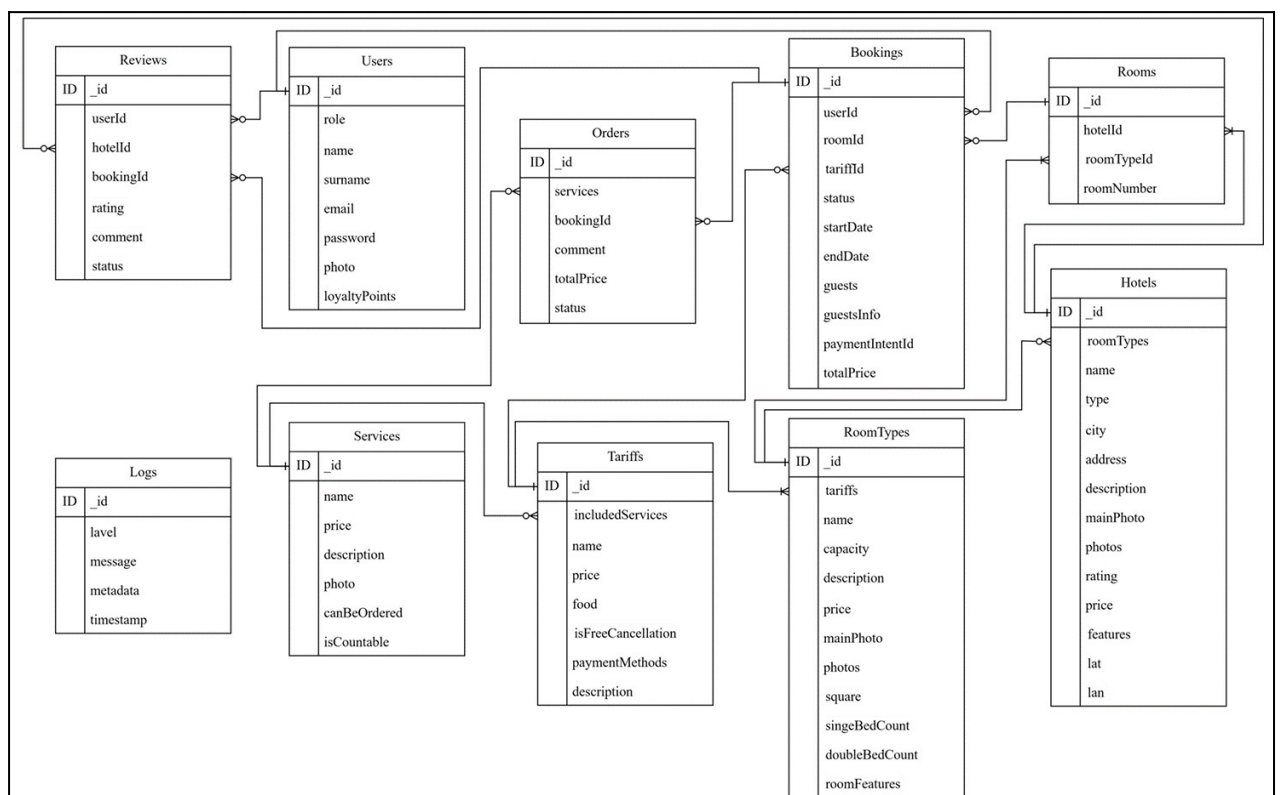


Рисунок 7 – Пример логической схемы базы данных (для не реляционной бд)

Блок-схемы:

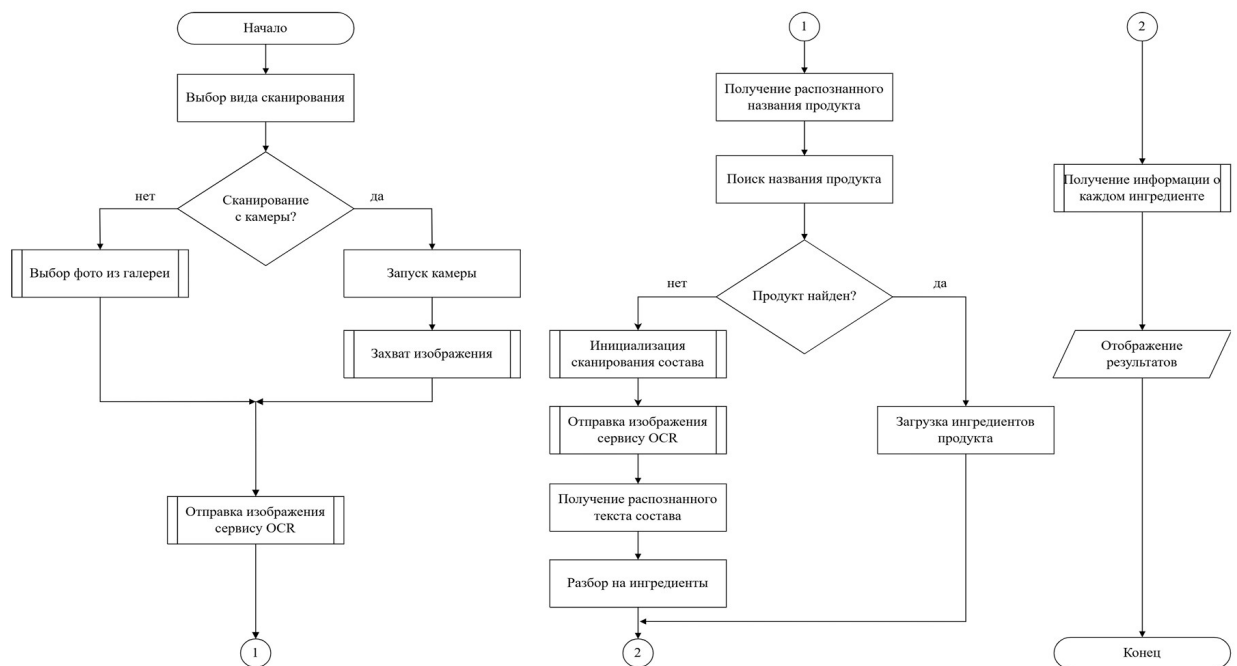


Рисунок 8 – Блок-схема алгоритма сканирования названия продукта и анализ его состава

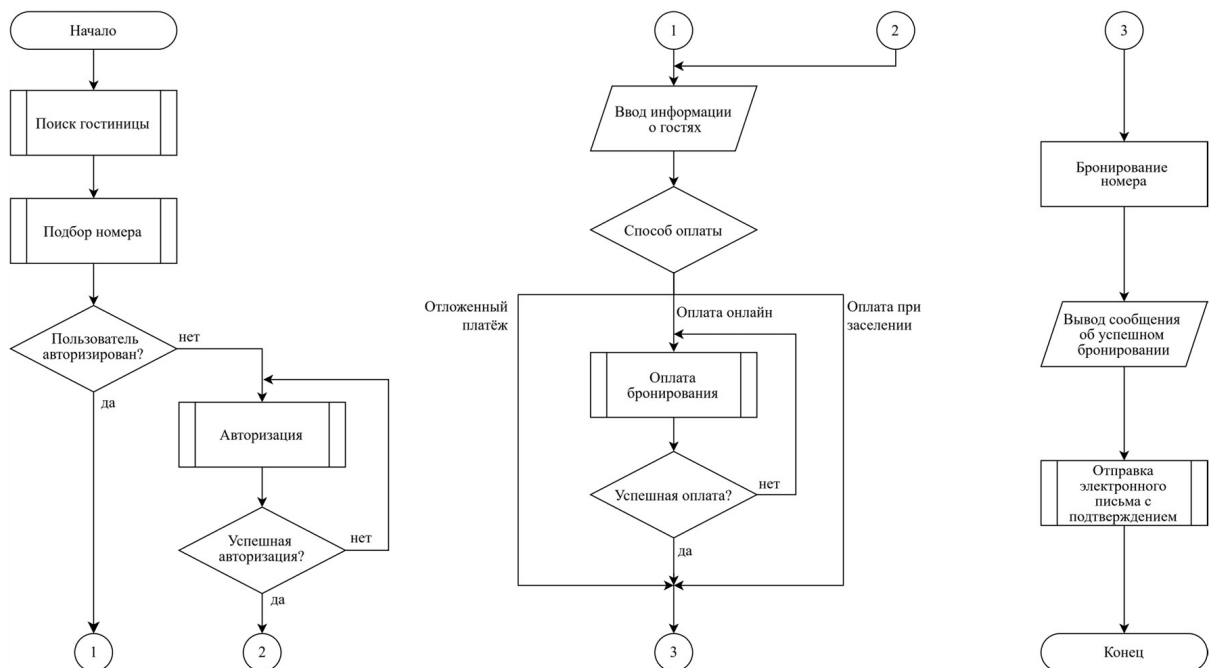


Рисунок 9 – Блок-схема алгоритма бронирования номера

Диаграмма последовательности: <https://habr.com/ru/articles/814769/>

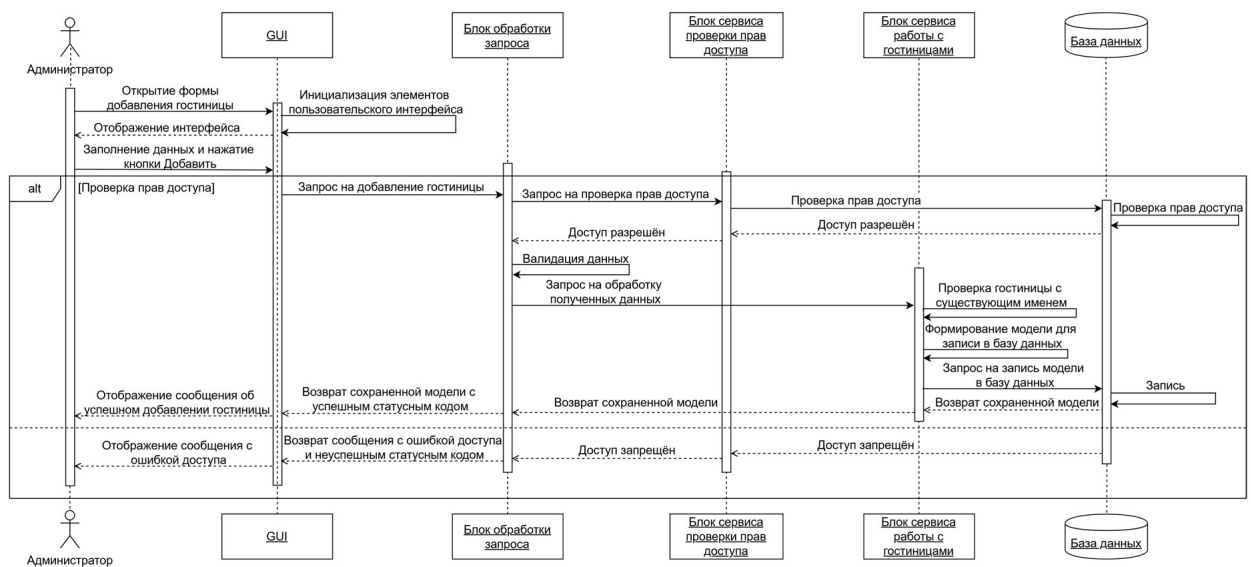


Рисунок 10 – Диаграмма последовательности добавления гостиницы

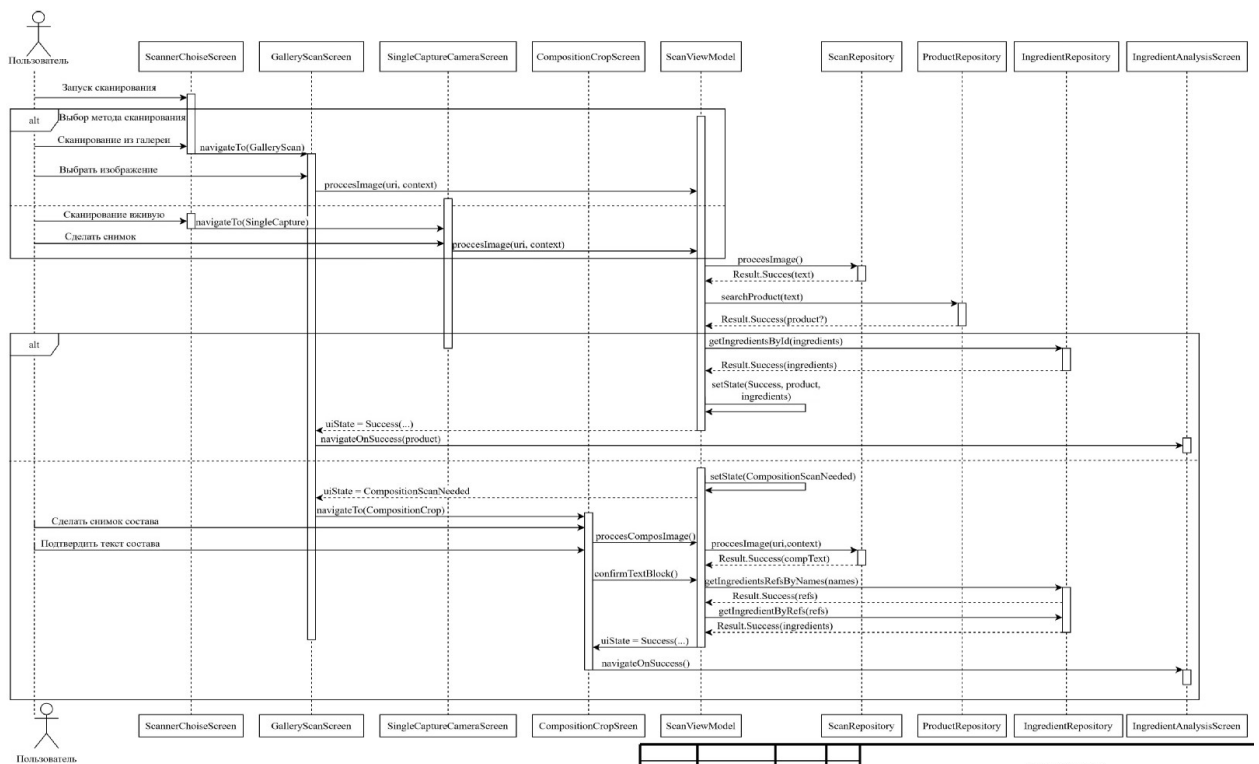


Рисунок 11 – Диаграмма последовательности алгоритма сканирования названия продукта и разбора его состава

Архитектура программы показывает модули (блоки) программного обеспечения и взаимодействия между ними. Как альтернатива может быть заменена схемой работы системы.

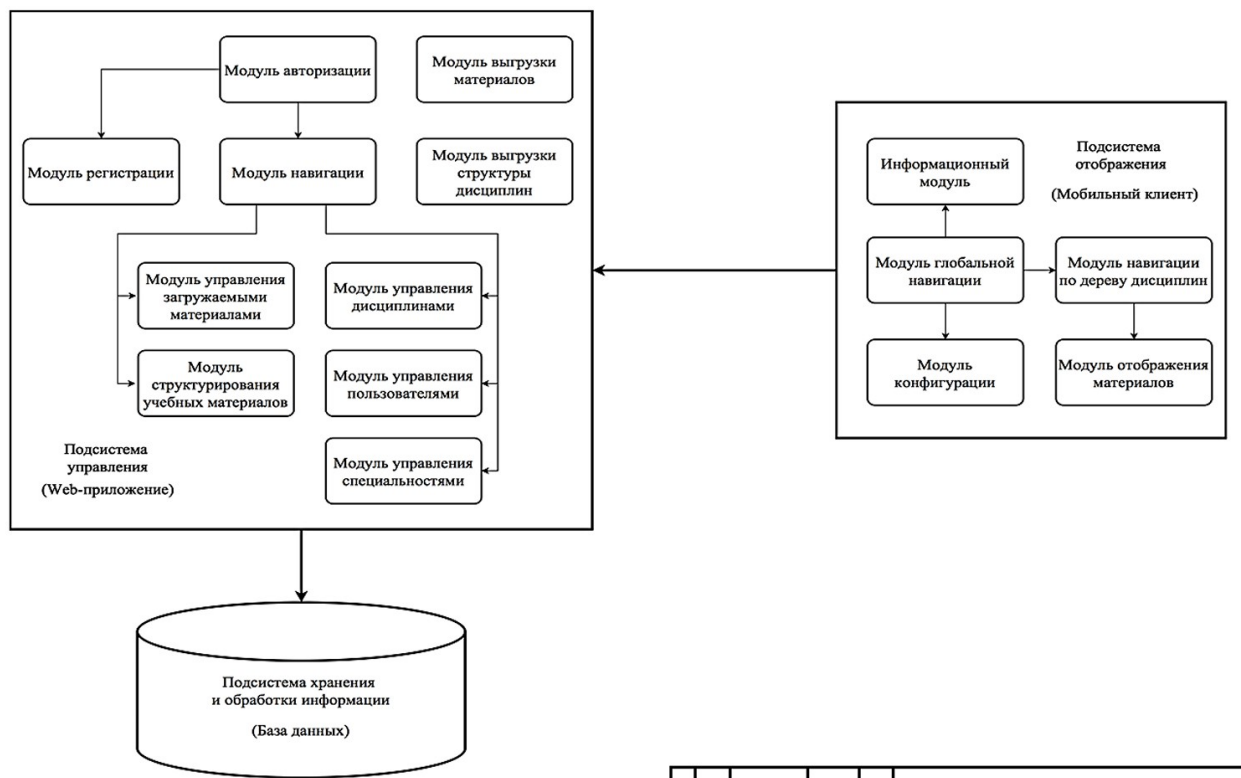


Рисунок 12 – Архитектура программы

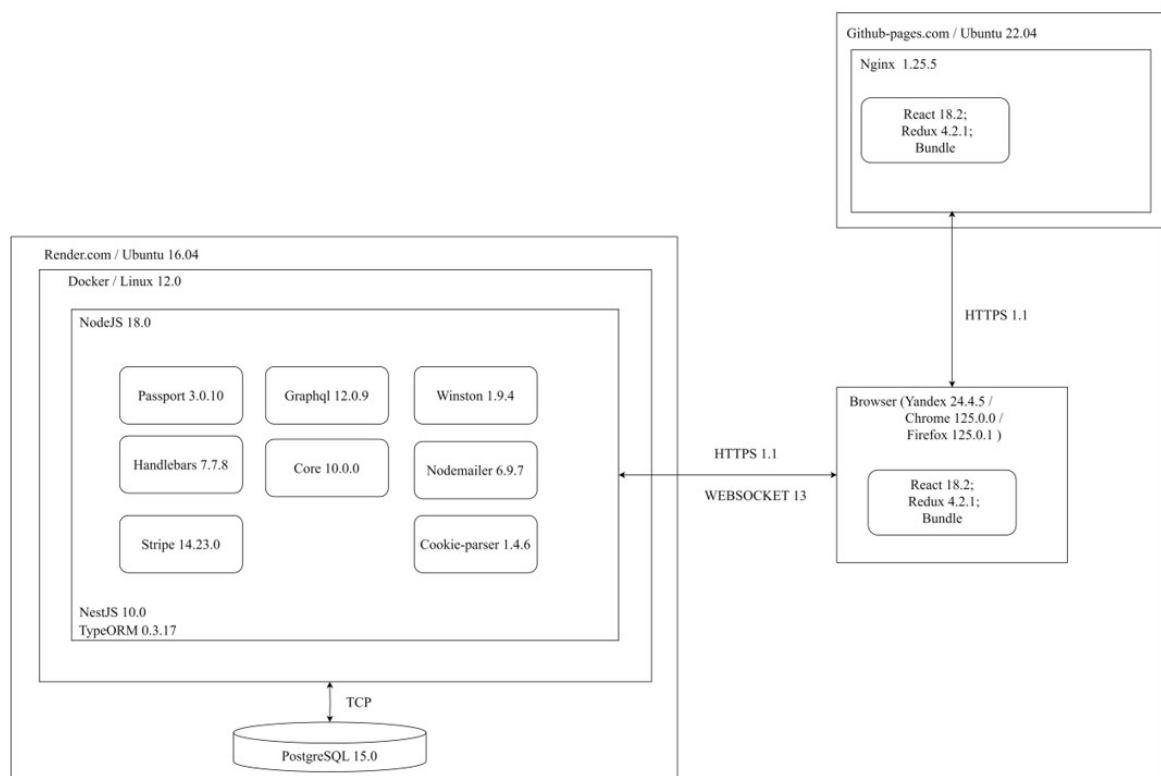


Рисунок 13 – Архитектура программы

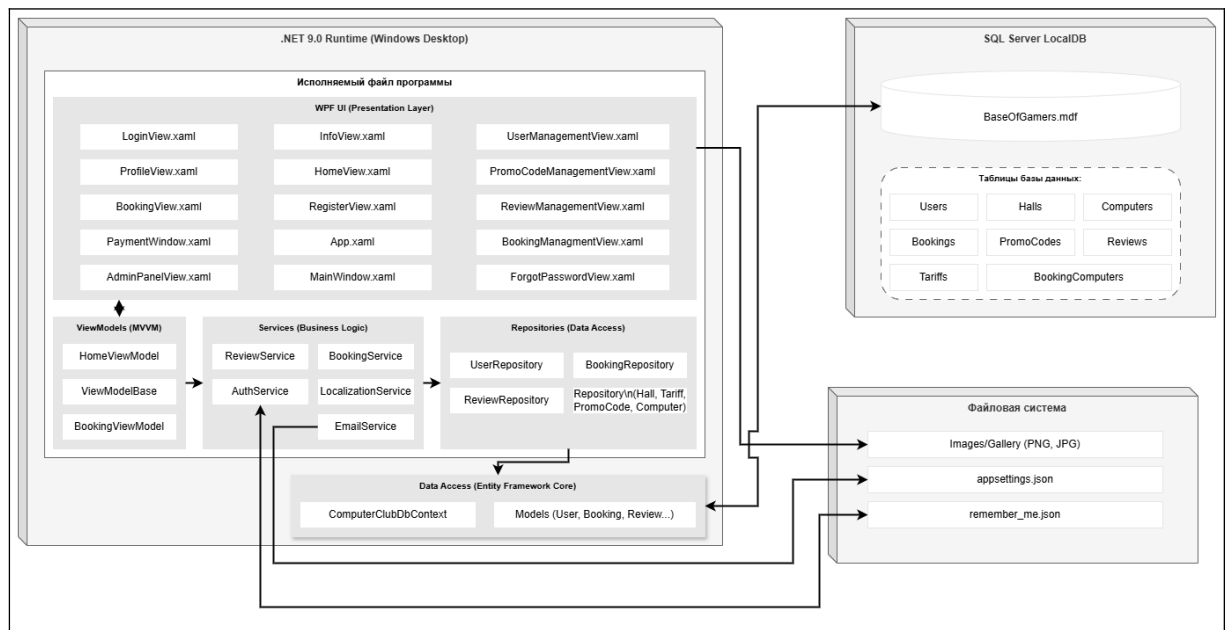


Рисунок 14 – Архитектура программы (все чертежи должны быть черно-белыми)

Диаграмма компонентов – статическая структурная диаграмма, которая показывает разбиение программной системы на структурные компоненты и связи (зависимости) между компонентами. В качестве физических компонент могут выступать файлы, библиотеки, модули, исполняемые файлы, пакеты и т. п.

Компонент – это физически существующая часть системы, благодаря которой обеспечивается реализация классов и отношений, а также достигается функциональность моделируемой ИС. Для компонентов UML определяет следующие стереотипы:

- file (файл) – самая распространенная разновидность компонента, принимающая вид какого-либо файла;
- executable (исполнимый) – разновидность файла, являющегося исполнимым; может исполняться на какой-либо компьютерной платформе;
- document (документ)
- разновидность файла в формате документа, который не является исполнимым и не содержит исходный код программы;
- library (библиотека) – разновидность файла в формате библиотеки (динамической или статической);
- source (источник) – разновидность файла, который содержит в себе исходный код программы;
- table (таблица) – разновидность компонента в формате таблицы БД.

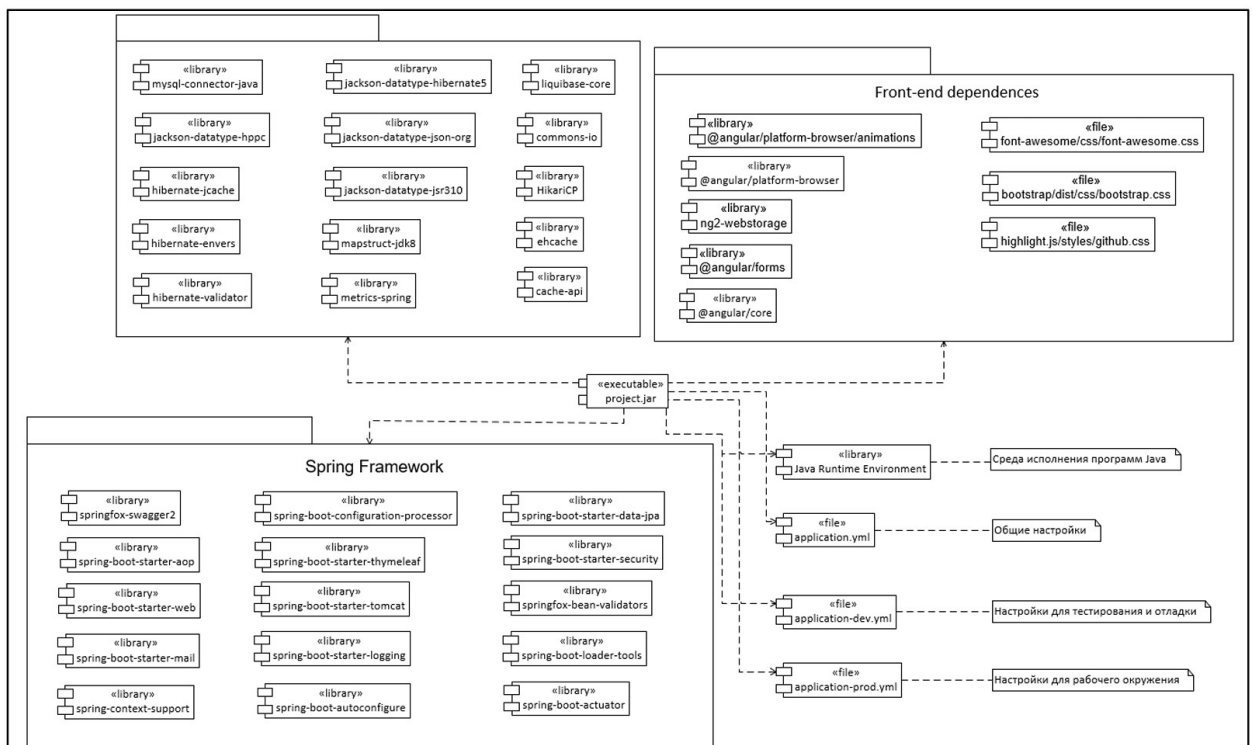


Рисунок 15 – Диаграмма компонентов

Диаграмма развертывания (синоним – диаграмма размещения). Она применяется для представления общей конфигурации и топологии распределенной программной системы и содержит распределение компонентов по отдельным узлам системы. Кроме того, диаграмма развертывания показывает наличие физических соединений – маршрутов передачи информации между аппаратными устройствами, задействованными в реализации системы. Диаграмма развертывания предназначена для визуализации элементов и компонентов программы, существующих лишь на этапе ее исполнения (runtime). При этом представляются только компоненты экземпляры программы, являющиеся исполнимыми файлами или динамическими библиотеками. Те компоненты, которые не используются на этапе исполнения, на диаграмме развертывания не показываются.

Составление и оформление диаграммы развертывания соответствуют диаграмме компонентов, поскольку компоненты описываются с использованием диаграмм развертывания.

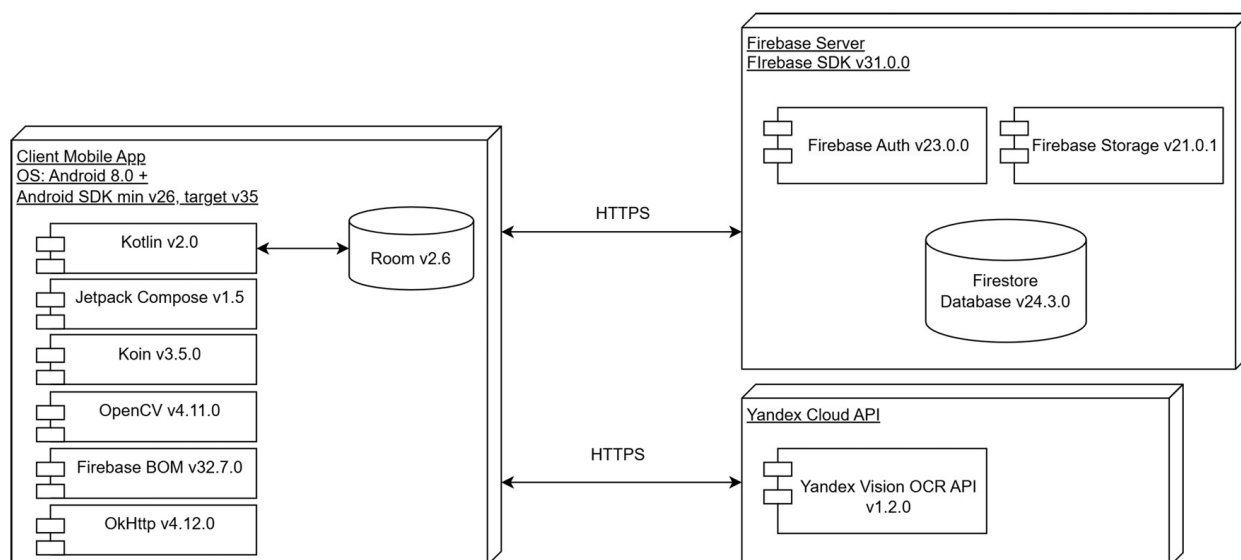


Рисунок 16 – Диаграмма развертывания мобильного приложения

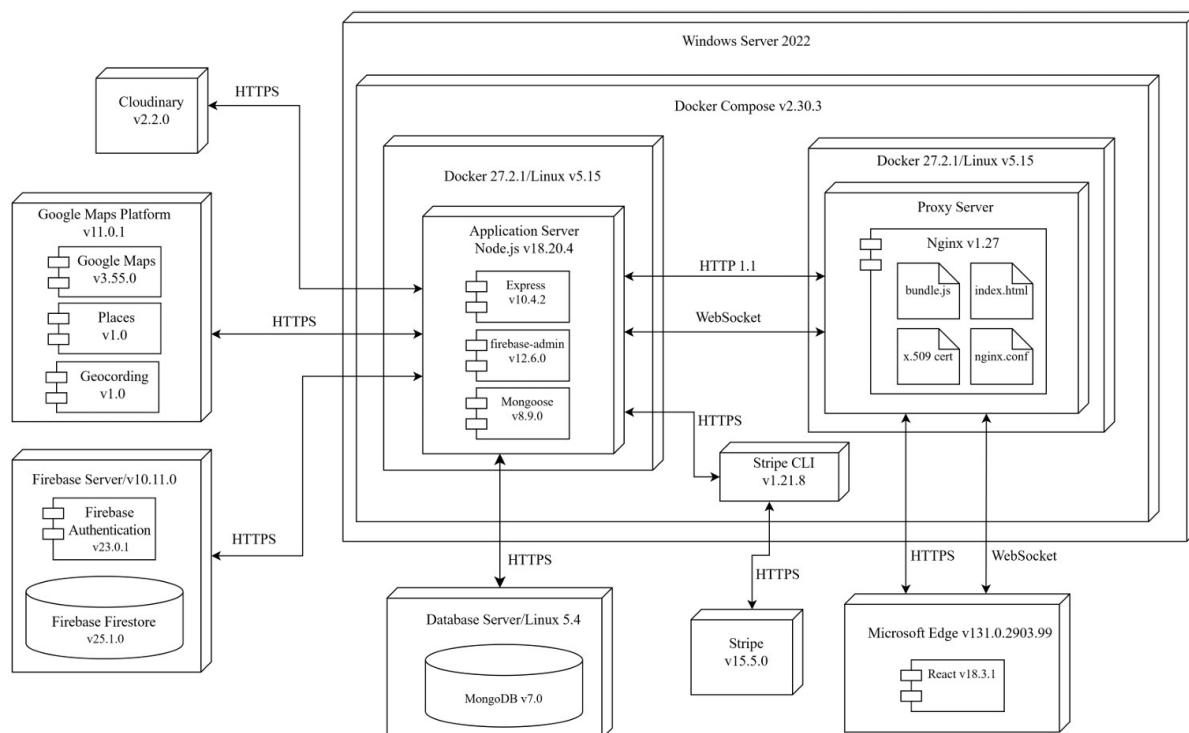


Рисунок 17 – Диаграмма развертывания веб-приложения